

Coze 开发与生产环境 技术参考 v2.0

developing-coze-apps 配套参考资料

Review baseline: 2026-08-17

适用：Coze Coding / 扣子编程全栈项目的开发、部署与维护

核心：DEV / PROD 环境隔离 · 数据库迁移 · 对象存储 · 环境变量 · Production Handoff

0. 阅读说明

这份 v2.0 将“开发环境说明”升级为“开发—部署—生产—维护”的工程参考。Coze 平台能力、CLI、模型和资源行为会演进，精确行为以当前官方文档、当前 CLI --help 与目标 workspace runtime 实测为准。

证据优先级：官方文档 > 当前 CLI/官方包 > 目标 workspace runtime > 实测报告 > 项目自定义实现。

1. 为什么必须区分开发环境与生产环境

开发阶段可以快速迭代，但生产环境承载真实用户、权限与业务数据。核心原则：同一套应用代码可以运行在 DEV 与 PROD，但 DB/Auth/Object Storage/Secrets/Business Data 默认按环境隔离。跨环境推进的是代码、Schema Migration 和显式批准的数据/资产。

默认策略：Schema 通过 migrations 推进；普通 DEV 测试/业务数据不自动进入 PROD。

2. DEV / PROD Environment Matrix

资源	DEV	PROD	Ownership	默认推进策略
Runtime/Project	开发预览/沙箱	正式部署	platform/user	同一 reviewed commit
Database	DEV DB	PROD DB	platform/app	migrations
Auth	DEV tenant/users	PROD tenant/users	platform/app	独立初始化
Object Storage	DEV bucket	PROD bucket	platform/app	asset allowlist/manifest
App Secrets	开发值	独立生产值	app_private	分别配置
Browser Public Config	DEV public config	PROD public config	client_public	构建/运行时核验

任何使用数据库、Auth、对象存储或 Secret 的项目，在写持久化业务前先建立 Environment Matrix。未知项标记 needs_runtime_check，不能猜。

3. 环境变量四分类

platform_injected：由 Coze/managed integration 在当前环境提供，例如实测环境曾出现 COZE_PROJECT_ENV、COZE_SUPABASE_*、COZE_BUCKET_*。生产部署先验证平台注入，不复制 DEV 值。

app_private：项目自有服务端 Secret，例如 API_KEY_HASH_PEPPER、第三方 API Secret、项目自定义 Bootstrap Token。DEV/PROD 独立配置。

client_public：明确允许进入浏览器的配置，例如 NEXT_PUBLIC_*。前缀意味着“浏览器可见”，不意味着“安全”。Service Role、Private Key、Bootstrap Token 绝不能放这里。

local_only：.env.local、mock endpoint、测试管理员等，只用于开发/本地，不是生产 source of truth。

4. Build-time 与 Runtime 配置

Next.js 等框架可能在 build 阶段把 client-public 变量写入浏览器 bundle。Server runtime 已经连到 PROD，不代表 Browser bundle 一定连到 PROD。部署后同时检查 browser config、server runtime config 与 CSP connect-src。

浏览器需要安全公开的运行时配置时，可采用 server runtime → allowlisted runtime-config/BFF → browser，只返回明确 client-safe 的字段。

5. 数据库：Schema 与 Data 分离

推荐生产推进逻辑：Code → same reviewed release；Schema → versioned migrations；Reference → reviewed idempotent seed/import；Business → no DEV sync by default；Auth users/sessions → no DEV sync by default。

如果平台部署 UI

提供“同步开发环境数据至生产环境”，它只是传输能力。默认关闭，除非用户明确要求并完成数据范围审核。

6. Migration 与 Seed

每个 Schema 变更进入版本控制中的有序 migration；执行前确认生产目标 DB identity；destructive migration 前备份并记录 rollback/restore；migration 与大规模业务导入分开。

Seed 分类：reference（审核后可进入 PROD）；demo（不进入）；privileged/default admin（不应普通 seed）；migration-support（显式 backfill/import）；business（默认不从 DEV 推进）。

7. Auth 与 First Admin / Bootstrap

BOOTSTRAP_TOKEN、BOOTSTRAP_ADMIN_EMAIL、/api/auth/bootstrap 等通常属于项目自定义机制，不是 Coze 通用规范。安全顺序：确认 PROD DB/Auth → production migrations → 配置项目自有 bootstrap secret → 显式调用一次 → 验证 admin identity/role → 限制/关闭/轮换 bootstrap 控制。

禁止把 .env.local 中的 DEV 默认管理员、已知默认密码或 DEV seed 管理员带入生产。

8. 对象存储

文件本体存对象存储；数据库持久化 object key/file ID，不持久化临时 signed URL。展示/下载时再生成当前可用 URL。

DEV 数据库里的 key 不代表 PROD bucket 已存在对应 object。需要上线的封面、模板图、课程素材使用 asset manifest/allowlist 显式推进，并在 PROD 验证 target key、content type、size/checksum 和 read behavior。

9. Coze CLI 与当前官方资料

Coze CLI 演进较快。执行生产操作前至少运行：coze --version；coze code --help；coze code env --help；coze code db --help。精确命令以当前安装版本与官方文档为准。

Canonical references：docs.coze.cn/guides_vibe_coding_overview；guides_internal_integrations；guides_integrate_database；guides_integrate_storage；guides_environment_variables；guides_integrate_authentication；npmjs.com/package/@coze/cli。

```
coze --version
coze code --help
coze code env --help
coze code db --help
```

10. Production Deployment 标准流程

1) Freeze release commit；2) source audit + environment audit；3) DEV/PROD Matrix；4) verify/create/bind PROD DB/Auth/Object Storage；5) 配置 app-private PROD secrets；6) destructive migration 前备份；7) 执行 migrations 并记录结果；8) 部署 reviewed commit/build；9) production smoke test；10) 如项目需要再执行 first-admin/bootstrap；11) 完成 bootstrap hardening；12) handoff 与 rollback 记录。

11. Production Smoke Test

至少验证：页面/health；登录/退出/session；RBAC 拒绝；disposable CRUD；disposable object 上传/读/删；AI normal request + handled failure；runtime public config 不泄漏 Secret；CSP/connect-src/frame policy；日志无 Secret；不存在意外 DEV/default admin。

12. 常见故障诊断

PROD 出现 DEV 管理员：检查 DEV data sync、seed/startup、.env.local、bootstrap 是否在生产变量准备前执行。

Server 已 PROD，Browser 仍 DEV：检查 NEXT_PUBLIC/build-time、browser bundle、runtime-config endpoint、CSP connect-src。

Terminal 修改了错误数据库：redeploy/resource 变化后不要信任旧 shell env；执行 privileged DB/Auth 操作前重新验证 production identity。Supabase-style 项目可运行 check_supabase_consistency.py。

迁移 DB 后图片丢失：数据库 key 不会复制 object；检查 PROD bucket 与 asset promotion。

Privileged operation 失败：检查 service/admin key 是否缺失，不要静默 fallback 到 anon/public credential；生产 fail fast。

13. Relay Studio 实战复盘

Relay Studio 的生产加固过程验证了这些可复用原则：DEV 默认账号/.env.local 不进入生产初始化；server runtime 与 browser build-time 配置分开；redeploy 后 terminal 可能指向旧资源；DB object key 与 storage object 要分别推进；production privileged config 缺失时 fail fast；Bootstrap 是项目机制。

Reference: github.com/douknowai/relay-studio

14. Production Handoff

交付记录应包含 release commit、production URL/project、DB/Auth/Bucket 安全标识、变量名称+ownership（不写 secret value）、migration version、backup/rollback、bootstrap 状态、smoke test、known risks、secret rotation/operation owner。

15. 最终原则

1. 同一套代码，不等于同一套数据。2. Schema migration $\neq$ DEV business-data sync。3. DEV object key $\neq$ PROD object 已存在。4. .env.local 不属于生产。5. NEXT_PUBLIC_* 不可以放 Secret。6. 平台注入变量先核验，不复制 DEV 值。7. Privileged production config 缺失时 fail fast。8. Bootstrap 先看项目代码，不假设 Coze 通用。9. 精确模型/CLI/配额/变量执行前重查。10. 生产修改前先确认当前操作的是哪一个环境/资源。

配套 Skill: <https://github.com/techdou/developing-coze-apps>